SUBMISSION DRAFT

RESCIENCE C

Replication / Computer Science

[Re] A Case for Partitioned Bloom Filters

Tejesh Vaish[1], ID

[1] Independent researcher, Dubai, United Arab Emirates

Edited by
(Editor)

Received
–

Published
–

DOI
–

Abstract. A Bloom filter can have a low average false-positive rate while returning many false positives for particular keys. This partial replication of Almeida's finite-size analysis recomputes 124 cells from four tables and matches 99 at the reported precision. The largest discrepancies are at low load. A bound rules out the choice of a common positive integer insertion count as an explanation for four mean-rate gaps at 64 bits. At 512 bits and eight probes, exact enumeration finds a 49.81 percent false-positive rate for zero-step queries under standard double hashing. Partitioning removes step-dependent variation in this model and raises the average rate from 0.649 to 1.442 percent. A partial replication of Almeida [1].

1 Introduction

A Bloom filter gives a fast answer to a membership query. It can return a false positive for an absent key. The probability usually reported is an average over absent keys. If one key is tested against many filters, the probability for that key matters instead. In a standard Bloom filter, several hash probes can land on the same bit. A repeated probe leaves one fewer bit for the query to check.

Almeida [1] compares standard and partitioned Bloom filters at finite sizes. This replication covers four tables and two curves in Sections 3 and 4, plus the three double-hashing mechanisms in Section 5. An exact extension computes how often the double-hashing mechanisms produce false positives under a uniform hash-pair model.

2 Methods

A standard filter has one array of m bits. Each insertion sets k positions and a query is positive if all its positions are set. In a partitioned filter, each of k equal parts has $s = m/k$ bits and receives one position per key. A collision between two positions within the index range of a part leaves two different physical bits because the positions belong to different parts.

2.1 Independent hashing

Fix a query that requires d distinct physical bits. After t insertion probes, let $p_t(j)$ be the probability that exactly j of those bits have been set. Start with $p_0(0) = 1$. A new independent uniform probe sets a previously unset query bit with probability $(d-j)/m$. The probability of satisfying the query after kn probes is $p_{kn}(d)$. This gives the same quantity as Almeida's conditional false-positive probability, using a recurrence on the number of covered query bits.

The program runs one chain with $d = k$. For a query that needs d bits, it averages $\binom{j}{d}/\binom{k}{d}$ over the chain's final state. The ratio is the chance that the d required bits are among the

Article license Creative Commons Attribution 4.0 International.
Correspondence should be addressed to Tejesh Vaish (t@tejeshvaish.com)
Submission draft. Publication details are assigned during editorial processing.
Code is available in the GitHub repository.

j covered bits. To obtain the global rate, the program averages over the number of distinct positions in k independent query probes. The partitioned rate under independent hashing is $[1-(1-1/s)^n]^k$. Each part has one required position and its insertion draws are independent of the other parts. The approximate standard formula in the source raises expected bit occupancy to the kth power.

2.2 Naive double hashing

Two uniform residues (a, b) generate the sequence $a + ib$ modulo m, or modulo s inside each partition. A standard filter treats the sequence as an unordered set of bits. A partitioned filter keeps the index i as part of the physical address.

To compute an exact rate for a fixed query, enumerate all possible insertion pairs. For each subset U of the query's required bits, count how many insertion pairs avoid every bit in U. The chance that all n insertions avoid U is the nth power of that fraction. Summing over U with sign $(-1)^{|U|}$ gives the chance that every required bit has been set. The calculation retains dependence among the k bits within each insertion pair.

2.3 Comparison protocol

For m bits, k probes, and a fraction f of nominal capacity, the primary analysis uses $n = \lfloor f(m/k) \log 2 \rfloor$ insertions. The source gives the nominal capacity as a real number. The printed F_a values in Table 1 identify the floor count in all eight nominal-load configurations. For reduced load, the floor is an explicit interpretation. A computed cell matches if it rounds to the published value, allowing 10^{-12} for numerical error at the boundary.

An integer inclusion-exclusion calculation checks the recurrence across the table grid. Tiny exhaustive enumerations check both layouts under independent and double hashing. The direct simulation builds 120,000 independent filters for each load at $m = 64, k = 4$. Within each filter it tests one fixed query for each distinct-bit count, from one to four. At full load it also tests a fresh random query per standard filter and runs a separate set of 120,000 partitioned filters. Wilson intervals describe each marginal probability. The queries share filters within a trial, so the intervals are neither independent nor simultaneous.

3 Average false-positive rates

At nominal capacity, the independent calculation matches all 24 F_a, F_s, and F_p cells in Almeida's Table 1 at their printed precision. Seven of the eight ratios match. The exception is $m = 64, k = 8$, where the paper gives 1.21703762 and the exact calculation gives 1.21703636. The reconstructed size sweep (Figure 1) shows the same broad pattern as the source's Figure 3. The ratio approaches one as the filter grows. With $k = 16$, the ratio is about 1.098 at $m = 512$ and about 1.011 at $m = 4096$.

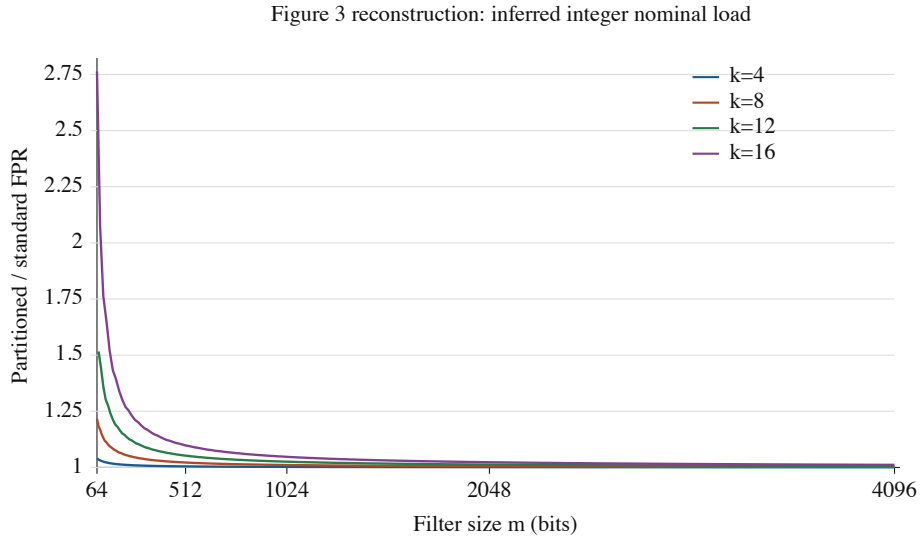


Figure 1. Reconstruction of Almeida's Figure 3. Each curve compares the exact global rates under independent hashing. The grid includes sizes divisible by k, with nominal capacity rounded down.

The reduced-load differences are larger. Table 1 shows the four largest Table 2 discrepancies, all at $m = 64$. Seven of the 16 reduced-load values in the source table differ at the printed precision. The eighth mismatch in the table is the nominal-load ratio repeated from Table 1. Switching from floor to nearest or ceiling reconciles none of the printed exceptions.

k	Load	n	Source Table 2	Recomputed	Upper bound
4	1/4	2	4.42059447	1.05697493	1.09842
4	1/2	5	1.11720759	1.04791623	1.09842
8	1/4	1	8.91227883	1.34006467	1.54612
8	1/2	2	1.77601565	1.28326972	1.54612

Table 1. Reduced-load ratios at $m = 64$. The last column bounds the ratio for any common positive integer insertion count under the paper's formulas.

No choice of a common positive integer insertion count produces these four ratios. Set $a = (1-1/m)^k$ and $b = 1-k/m$. Then $0 \le b \le a < 1$. For every integer $n \ge 1$, the lower bound $F_a \le F_s$ gives

$$\frac{F_p}{F_s} \le \frac{F_p}{F_a} = \left(\frac{1-b}{1-a}\frac{\sum_{j=0}^{n-1} b^j}{\sum_{j=0}^{n-1} a^j}\right)^k \le \left(\frac{1-b}{1-a}\right)^k.$$

The bound is 1.09842 for four probes and 1.54612 for eight. The reported half- and quarter-load ratios exceed these bounds. The source formulas leave the cause of the difference unresolved.

4 False positives for a fixed key

The conditional rate rises as repeated probes reduce the number of distinct bits the query needs. Thirty-two of the 48 cells in Almeida's Table 3 match, including all 16 at nominal load. All 20 cells of Table 4 match. The mismatches in Table 3 occur at reduced

load. For example, at $m = 64, k = 4$ and quarter load, a one-bit query has a rate 581.88 times the global average in this calculation, compared with 804.25 in the paper. In the reconstructed model the one-bit rate is 11.84%, and the global rate is about 0.0203%. The tail plot (Figure 2) reconstructs the source's Figure 4. For $m = 512, k = 16$ at nominal load, at least three collisions occur for about one query in 914. For these queries, the false-positive rate is at least 6.78 times the global rate. At least five collisions occur for about one in 1.34 million, with a rate of about 28.30 times the global rate. These thresholds refer to collision classes of absent queries under independent uniform hashing.

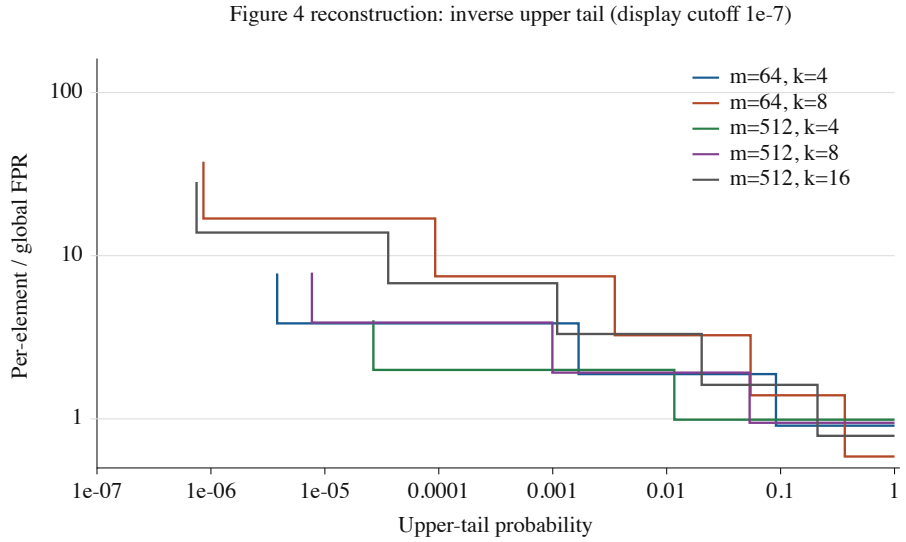


Figure 2. Reconstruction of Almeida's Figure 4. Each point shows the cumulative query probability at or above the corresponding fixed-key/global rate ratio. The plot omits tail probabilities below 10^{-7}.

The direct simulations met the prespecified calibration criterion. The largest absolute standardized error was 1.61 among 13 eligible cells, below the cutoff of 4.5. Table 2 shows selected nominal-load results. The experiment checks the predicted rates at this configuration. The global estimates have overlapping intervals.

Query	Hits	Estimate	95% interval	Model
Standard, one bit	59728	0.49773	[0.49490, 0.50056]	0.49989
Standard, four bits	6959	0.05799	[0.05668, 0.05933]	0.05833
Standard, random	7736	0.06447	[0.06309, 0.06587]	0.06423
Partitioned, random	7980	0.06650	[0.06510, 0.06792]	0.06676

Table 2. Selected direct simulations at $m = 64$, $k = 4$, and $n = 11$. Each row uses 120,000 trials. The intervals are marginal Wilson intervals.

5 Double hashing

At $m = 512, k = 8$, a standard filter with step $b = 0$, 256, or 128 touches one, two, or four physical bits, respectively. Using zero, half, or a quarter of the part size as the step in a partitioned filter still touches eight bits. The pairs $(7, 5)$ and $(42, -5)$ visit the same standard-filter bits in reverse order. Their partitioned addresses have no overlap. The pairs $(7, 5)$ and $(12, 5)$ share seven standard-filter positions and no partitioned positions. These three examples recover the mechanisms in Almeida's Figures 5–7.

The enumeration measures how these cases affect false positives. For a standard filter, translation removes the starting residue. Multiplication by an odd unit modulo 512 preserves the insertion-pair law and maps steps with the same greatest common divisor to each other. Each step class thus has one fixed-key rate. For a partitioned query with pair (a_q, b_q), subtracting the query's pair from each insertion pair maps the query to $(0, 0)$ while preserving the insertion law. Every fixed query therefore has the same rate in this model.

Layout	Representative step	Bits	Class mass	FPR (%)
Standard	1	8	1/2	0.441
Standard	64	8	1/128	0.918
Standard	128	4	1/256	6.363
Standard	256	2	1/512	24.717
Standard	0	1	1/512	49.810
Partitioned	Any	8	1	1.442

Table 3. Selected exact double-hashing classes for $m = 512$, $k = 8$, and $n = 44$. Each standard row represents all steps with the same $\gcd(b, m)$. The artifact includes every class.

At $n = 44$, the zero-step standard query has a 49.81% false-positive rate and accounts for one in 512 hash pairs. The average standard double-hashing rate is 0.649%. Every fixed partitioned query has a 1.442% rate, which also gives its global rate. Partitioning removes the step-dependent variation in this model and raises the average at this configuration. The 0.390% partitioned result under independent hashing uses a different joint distribution of insertion probes.

6 Scope and availability

The independent implementation recovers the nominal-load comparison and the concentration of false positives among keys with repeated probes. At low load, several printed ratios remain unresolved. The artifact preserves each source value beside the computed value, along with the model and integer count used for the comparison.
The results concern uniform hash outputs at finite filter sizes. The experiment does not measure deployed hash functions, workload-dependent key frequencies, SIMD throughput, or the applications in Sections 6 and 7 of the source. The double-hashing extension covers one allocation and one load. The comparison values come from the public accepted manuscript, arXiv:2009.11789v2. The original author's code and experiments were not used, and the author has not been contacted about the discrepancies.
The companion artifact contains the code, transcribed values, and generated tables and plots. A single Python command reruns the experiment offline. The PDF build reads the generated results.

References

1. P. S. Almeida. "A Case for Partitioned Bloom Filters." In: **IEEE Transactions on Computers** 72.6 (2023), pp. 1681–1691. DOI: 10.1109/TC.2022.3218995. arXiv:2009.11789v2.